

فایل های تکراری

- محدودیت زمان: ۱ ثانیه
- محدودیت حافظه: ۲۵۶ مگابایت

هدف از این برنامه، پیمایش بازگشتی در یک پوشه‌ی والد مشخص شده (Parent Folder) و تمام زیرپوشه‌های آن و تهیه لیستی مرتب از فایل‌هایی است که دارای نام و محتوای یکسان (Duplicated Files) هستند.

فایل های تکراری را فایل هایی در نظر می‌گیریم که هردو شرط زیر را داشته باشند:

- **نام یکسان:** نام فایل‌ها (بدون در نظر گرفتن مسیرشان) کاملاً یکسان باشد.
- **محتوای یکسان:** محتوای داخلی فایل‌ها یکسان باشد، که این امر از طریق مقایسه‌ی کد هش محتوا (Content Hash Code) بررسی می‌شود.

هش کردن محتوا:

برای بررسی یکی بودن محتوای فایل، از آنجا که بررسی کل محتوای هر فایل با محتوای هر یک از فایل های دیگر زمان بر است، ابتدا محتوای فایل را هش می‌کنیم و سپس آن را بررسی می‌کنیم.

مفهوم هش کردن (Hashing) :

هش کردن (Hashing) فرایندی است که در آن یک داده‌ی ورودی با هر اندازه (مثل یک رشته، عدد یا فایل) از طریق یک تابع ریاضی خاص به یک مقدار خروجی با اندازه ثابت تبدیل می‌شود که به آن کد هش (Hash Code) یا Digest می‌گویند.

به همین جهت طولانی بودن محتوای فایل با هش کردن محتوای آن دیگر مشکل ساز نخواهد بود.

هش کردن در زبان C:

استفاده از کتابخانه‌های آماده (برای هش امن مثل SHA یا MD5)

برای هش محتوای فایل، باید از الگوریتم‌هایی مثل SHA-1، MD5، یا SHA-256 استفاده کرد. در C این‌ها را می‌توان با کتابخانه‌هایی مثل زیر استفاده کرد:

- <openssl/md5.h>
- <openssl/sha.h>

مثلاً برای SHA256:

```
1  #include <stdio.h>
2  #include <openssl/sha.h>
3
4  int main() {
5      FILE *file = fopen("example.txt", "rb");
6      if (!file) return 1;
7
8      unsigned char data[1024];
9      unsigned char hash[SHA256_DIGEST_LENGTH];
10     SHA256_CTX ctx;
11
12     SHA256_Init(&ctx);
13
14     size_t bytes;
15     while ((bytes = fread(data, 1, 1024, file)) != 0)
16         SHA256_Update(&ctx, data, bytes);
17
18     SHA256_Final(hash, &ctx);
19
20     fclose(file);
21
22     printf("SHA256: ");
23     for (int i = 0; i < SHA256_DIGEST_LENGTH; i++)
24         printf("%02x", hash[i]);
25     printf("\n");
26
27     return 0;
28 }
```

ساختار پیشنهادی :

پیشنهاد میشود از struct برای نگهداری اطلاعات فایل‌ها استفاده شود.

محدودیت‌ها

کتابخانه های مجاز در این تمرین برای کار با فایل‌ها: `stdlib.h * string.h * dirent.h` برای هش کردن:

`openssl/sha.h`

ورودی

آدرس فولدر parent مربوطه که باید برنامه در تمام پوشه های آن جستجو کرده و فایل های تکراری را لیست کند. **توجه:** آدرس به عنوان ورودی `argv` هنگام کامپایل داده می شود.

▼ توضیحات

وقتی تابع `main` را به صورت `int main(int argc, char *argv[])` می‌نویسیم، دو ورودی تابع `main` به صورت زیر است: `argc` (تعداد کلمات): مخفف Argument Count است. یک عدد صحیح است که می‌شمارد چند کلمه در خط فرمان تایپ شده است. (**توجه:** خود اسم برنامه هم همیشه شمارش می‌شود، پس `argc` حداقل ۱ است).

است. یک لیست (آرایه) از رشته‌هاست که کلمات تایپ شده Argument Vector مخفف: (خود کلمات) `argv` را به ترتیب در خود نگه می‌دارد.

برای ورودی دادن به برنامه اجرایی مان، به روش زیر عمل می‌کنیم:

```
gcc main.c -o out -lssl -lcrypto
```

(1- مربوط به کتابخانه هایی می‌شود که لازم داریم در هنگام کامپایل استفاده شود.)

از آنجا که در این مثال از کتابخانه `openssl` برای هش استفاده می شود، لازم است این flag ها را تنظیم کنیم.

مرحله بعد (اجرا):

```
./out my_folder_name
```

که در اینجا نام فایل اجرایی `out` است و `argc = 2` است و `argv = ["my_folder_name"]`

نسخه اولیه تابع `main`:

```
int main(int argc, char *argv[]) {  
  
    if (argc < 2) {
```

```

4         printf("Error: Please provide a folder name.\nUsage: %s <folder>");
5         return 1;
6     }
7
8     // your code here
9
10    return 0;
11 }

```

خروجی

باید گزارشی مرتب شده از فایل های تکراری آماده شود.

توجه :

- فایل های تکراری به ترتیب الفبایی نامشان باید مرتب شود. (یعنی اولین فایل تکراری از نظر الفبایی از دومین فایل تکراری بالاتر باشد)
- و همچنین آدرس های مربوط به هر فایل، به ترتیب الفبایی مرتب شود.

فرمت خروجی:

در صورت وجود حداقل یک فایل تکراری:

Duplicated Files List:

1.(<File Name>)

<Path 1>

<Path 2>

...

2. (<File Name>)

<Path 1>

<Path 2>

...

و در صورت عدم وجود هیچ فایل تکراری:

Duplicated Files List:

نمونه:

ورودی

my_folder

خروجی

Duplicated Files List:

1.(a1.txt)

my_folder/home/user/data/a1.txt

my_folder/home/user/test/a1.txt

2.(bi.bin)

my_folder/root/bin/bi.bin

my_folder/root/bin/bin1/bi.bin

my_folder/root/bi.bin

که در این مثال ساختار فولدر my_folder به شکل زیر است:

```
main.c
my_folder/
├─ unique.log
├─ doc.txt
├─ root/
│   ├── bi.bin
│   └─ bin/
│       ├── bi.bin
│       ├── bin1/
│       │   └─ bi.bin
│       └─ bin2/
│           └─ log_file.txt
└─ home/
    └─ user/
        ├── data/
        │   └─ a1.txt
        └─ test/
            ├── a1.txt
            └─ doc.txt
```

توجه: doc.txt ها محتوای متفاوتی دارند بنابراین در لیست تکراری ها نمایش داده نمی‌شوند.

شبیه سازی پایگاه داده

- محدودیت زمان: ۱ ثانیه
- محدودیت حافظه: ۲۵۶ مگابایت

مقدمه

در این تمرین می خواهیم با استفاده از ساختارها (struct) و فایل های باینری، مفهوم ساده ای از پایگاه داده (Database) را شبیه سازی کنیم.

پایگاه داده در واقع مجموعه ای از رکوردها (Records) است که هر رکورد شامل چند فیلد (Field) می باشد. همچنین ذخیره و بازیابی اطلاعات دانشجویان در فایلی به نام database.dat باشد به طوری که اطلاعات با بسته شدن برنامه از بین نروند.

مشخصات داده ها

هر رکورد دانشجو شامل فیلدهای زیر است که باید در یک struct تعریف شوند:

۱. شناسه (ID): عدد صحیح (تولید خودکار توسط برنامه).

۲. نام (Name): رشته (حداکثر ۵۰ کاراکتر).

۳. سن (Age): عدد صحیح.

۴. رشته تحصیلی (Field): رشته (حداکثر ۵۰ کاراکتر)

قوانین تولید شناسه (ID)

شناسه (ID) هر دانشجو باید به صورت خودکار محاسبه شود:

- برنامه باید آخرین رکورد موجود در فایل database.dat را بخواند.
 - شناسه رکورد جدید برابر است با: (شناسه آخرین رکورد) + 1.
 - اگر فایل خالی است یا وجود ندارد، شناسه اولین رکورد ۱ خواهد بود.
- نکته: این روش یعنی اگر آخرین رکورد فایل حذف شود، شناسه آن ممکن است به رکورد بعدی اختصاص داده شود (چون مبنا همیشه انتهای فیزیکی فایل است)

در پایگاه داده طراحی شده باید که دستورات شبیه به دستورات SQL مانند موارد زیر را پشتیبانی شود:

- `INSERT` (افزودن داده جدید)،
- `SELECT` (نمایش داده‌های ذخیره شده)،
- `UPDATE` (ویرایش داده موجود بر اساس ID داده شده)،
- `DELETE` (حذف داده بر اساس ID داده شده) را شبیه‌سازی کند.

همچنین، برنامه باید تا زمانی که کاربر دستور `"exit"` را وارد نکرده، در یک حلقه‌ی تکرار ادامه پیدا کند.

توصیه: در برنامه‌نویسی، برای سازمان‌دهی و مدیریت این اطلاعات، معمولاً از **توابع (Functions)** استفاده می‌شود تا هر عمل خاص (مثل افزودن، نمایش، حذف یا ویرایش اطلاعات) در قالب یک تابع جداگانه انجام شود. به این ترتیب، برنامه ساختاریافته‌تر، خواناتر و قابل توسعه‌تر خواهد بود. بنابراین تابع‌های زیر را باید تعریف کنید.

ورودی و خروجی

1. افزودن رکورد یا INSERT :

فرمت دستور:

```
INSERT  
student(name, age, field)
```

برای نمونه

```
INSERT  
student(Amir, 20, Electrical engineering)
```

2. نمایش همه رکوردهای داخل فایل `database.dat` یا SELECT :

فرمت دستور:

```
SELECT
```


فرمت خروجی:

اول و انتها خروجی متناظر خط چین ها هستند و سطر دوم ID / name / age / field قرار دارد و داده های موجود به همین فرمت قرار می گیرند.

نکته: حتی اگر داده ای موجود نباشد، خط چین ها و ID / name / age / field نیز به همان ترتیب هستند ولی داده ای در آن میان نیست.

برای نمونه:

```
-----  
ID / name / age / field  
1 Mohammad Reza 20 Computer Engineering  
2 Fatemeh 19 Computer Engineering  
3 Ali 19 Mechanical engineering  
-----
```

3. ویرایش رکورد بر اساس ID یا UPDATE:

فرمت دستور:

```
UPDATE  
ID: student-ID  
student(NewName, NewAge, NewField)
```

برای نمونه

```
UPDATE  
ID: 21  
student(Sara, 22, Electrical engineering)
```

فرمت خروجی:

- در صورتی که این آیدی موجود نبود خروجی زیر را بدهد:

```
ID not found!
```

- اگر تغییر صورت گرفت خروجی زیر را بدهد:

Record updated.

4. حذف رکورد بر اساس ID یا DELETE:

دستور:

```
DELETE  
ID: student ID
```

برای نمونه

```
DELETE  
ID: 30
```

خروجی:

- در صورتی که این آیدی موجود نبود خروجی زیر را بدهد:

ID not found!

- اگر داده حذف شد خروجی زیر را بدهد:

Record deleted.

توجه کنید که نیازی نیست آیدی ها را شیفت کنند و یا دیگر آیدی ها تغییر کنند.

5. خروج :

برای پایان برنامه دستور زیر وارد شود

```
exit
```

نکات فنی برای پیاده سازی

۱. نام فایل: از نام database.dat استفاده کنید.
۲. فایل موقت: برای UPDATE و DELETE میتوانید از یک فایل کمکی (مثلاً temp.dat) استفاده کنید؛ رکوردهای باقیمانده را در آن بنویسید و سپس نام آن را به database.dat تغییر دهید.
۳. خواندن رشته های دارای فاصله: در دستوراتی مثل INSERT، نام و رشته تحصیلی ممکن است دارای فاصله باشند (مانند "Ali Reza"). استفاده از scanf با فرمتهایی مثل %[^\n,] (خواندن تا رسیدن به ویرگول) توصیه میشود

مثال

ورودی نمونه

```
INSERT
student(Amin, 20, Computer Engineering)
INSERT
student(Hossein, 19, Chemical Engineering)
INSERT
student(Zahra, 19, Mechanical Engineering)
SELECT
UPDATE
ID: 2
student(Hossein, 20, Computer Engineering)
SELECT
DELETE
ID: 1
SELECT
DELETE
ID: 1
exit
```

خروجی نمونه

```
-----
ID / name / age / field
1 Amin 20 Computer Engineering
2 Hossein 19 Chemical Engineering
3 Zahra 19 Mechanical Engineering
-----
```

Record updated.

ID / name / age / field

1 Amin 20 Computer Engineering

2 Hossein 20 Computer Engineering

3 Zahra 19 Mechanical Engineering

Record deleted.

ID / name / age / field

2 Hossein 20 Computer Engineering

3 Zahra 19 Mechanical Engineering

ID not found!

در مثال داده شده در ابتدا 3 عدد دانشجو اضافه شد. پس از نمایش اطلاعات، با دادن شناسه دانشجو اطلاعات مربوط به آن را تغییر داده. با حذف کردن یک دانشجو، شناسه دیگر دانشجو ها تغییر نکرد. صدا زدن دستور حذف کردن دانشجویی هم که شناسه اش دیگر موجود نیست خطای مربوط به آن داده شده. درنهایت نیز با دستور exit به برنامه پایان داده.

پلی لیست

- محدودیت زمان: ۱ ثانیه
- محدودیت حافظه: ۲۵۶ مگابایت

در این تمرین قصد داریم یک برنامه برای مدیریت پلی‌لیست آهنگ‌ها بنویسیم. هدف این است که با **فایل‌های باینری** کار کنیم، اطلاعات آهنگ‌ها را بخوانیم و تغییر بدهیم و بعد نتیجه‌ی کارهای انجام شده را در یک **فایل متنی** ذخیره کنیم.

توضیح مسئله

ما یک فایل باینری به نام playlist.dat داریم که اطلاعات آهنگ‌ها در آن نگهداری شده است. هر آهنگ شامل این ویژگی‌هاست:

- نام آهنگ (name)
- نام هنرمند (artist)
- مدت زمان آهنگ به ثانیه (duration)
- مشخصه اینکه آهنگ جزو علاقه‌مندی‌هاست یا نه (Favorite)

وظیفه شما این است که برنامه‌ای طراحی کنید که این **فایل باینری** را باز کند. (در صورتی که فایل از قبل وجود نداشت این فایل را بسازد). دستورات زیر را روی فایل باینری اجرا و ذخیره نماید و خروجی تمام عملیات‌ها را در یک فایل متنی به نام output.txt ثبت کند.

نکته : نحوه ذخیره آهنگ در فایل باینری به این شکل است (در واقع آرایه‌ای از Song ها) :

```
1 | typedef struct {
2 |     char name[NAME_LEN];
3 |     char artist[ARTIST_LEN];
4 |     int duration;        // in seconds
5 |     int isFavorite;
6 | } Song;
```

دستورات برنامه

۱. اضافه کردن آهنگ جدید:

```
add <name> <artist> <duration>
```

آهنگی را به انتهای لیست اضافه می کند و در فایل ذخیره می کند.

- مدت زمان (duration):
 - یا به صورت mm:ss (مثلاً 4:12)
 - یا یک عدد صحیح که نشان دهنده ی مدت زمان به ثانیه است. مطابق با دستور داده شده خروجی های زیر را به فایل تکست اضافه کنید:
- اگر دستور از نظر تعداد پارامترها درست نباشد:

```
Invalid add command.
```

- اگر فرمت مدت زمان نامعتبر باشد:

```
Invalid duration format: <duration>
```

- اگر همه چیز معتبر بود آهنگ جدید به انتهای فایل بایرنی اضافه می شود:

```
Song added: <songName> by <artist>
```

اگر آهنگی از قبل وجود داشته باشد، در صورت آمدن این دستور **باز هم اضافه می شود**

۲. حذف آهنگ یا آهنگ ها با نام مشخص:

```
delete <name>
```

- اگر حداقل یک آهنگ حذف شد:

```
Deleted: <name>
```

- اگر هیچ آهنگی با این نام وجود نداشت:

Song not found: <name>

- اگر چند آهنگ با یک نام وجود داشته باشد، همه‌ی آن‌ها حذف می‌شوند

۳. علامت‌گذاری آهنگ به عنوان علاقه‌مندی (favorite):

star <name>

اولین آهنگی که نامش برابر <name> است پیدا می‌شود و تغییرات در فایل باینری ذخیره می‌شود.

- اگر آهنگ پیدا شد:

Starred: <name>

- اگر آهنگ پیدا نشد هیچ خروجی‌ای تولید نمی‌شود و فایل باینری تغییر نمی‌کند

۴. برداشتن علامت علاقه‌مندی از آهنگ:

unstar <name>

- اگر آهنگ پیدا شد، علاقه‌مندی برداشته می‌شود و خروجی زیر به فایل تکست اضافه می‌شود:

Unstarred: <name>

- اگر آهنگ پیدا نشد هیچ خروجی‌ای تولید نمی‌شود و فایل باینری تغییر نمی‌کند

۵. نمایش همه آهنگ‌ها:

list

- اگر لیست خالی باشد:

Playlist is empty.

- در غیر این صورت، برای هر آهنگ یک خط با فرمت زیر نوشته می‌شود:

<name> | <artist> | <duration> | (Favorite|Normal)

- مدت زمان همیشه به صورت **ثانیه** چاپ می‌شود.
- ترتیب نمایش اولیه همان ترتیب ذخیره در فایل است.

۶. نمایش فقط آهنگ‌هایی که favorite هستند.

filter_favorites

- اگر هیچ آهنگی ستاره‌دار نبود:

No favorites found.

- در غیر این صورت، فقط آهنگ‌های favorite با فرمت زیر چاپ می‌شوند:

<name> | <artist> | <duration> | (Favorite|Normal)

۷. مرتب‌سازی آهنگ‌ها بر اساس نام آهنگ:

sort_name

معیار مرتب‌سازی

۱. ابتدا بر اساس name

۲. اگر برابر بود، بر اساس artist

- ترتیب جدید در فایل باینری ذخیره می‌شود
 - خروجی مطابق با دستور:

Sorted by name.

۸. مرتب‌سازی آهنگ‌ها بر اساس نام هنرمند:

sort_artist

معيار مرتب سازي

۱. ابتدا بر اساس artist

۲. اگر برابر بود، سپس name

+خروجی:

Sorted by artist.

۹. مرتب سازي بر اساس مدت زمان:

sort_duration

معيار مرتب سازي

۱. ابتدا duration (صعودی)

۲. سپس artist

۳. سپس name

Sorted by duration.

۱۰. امضای پلی لیست:

playlist_signature

هدف محاسبه ی یک عدد به عنوان امضای پلی لیست است. لین امضا برای تشخیص تغییرات در محتوای پلی لیست استفاده می شود.

روش محاسبه

فرض کنید پلیلیست شامل n آهنگ است و آهنگ‌ها به ترتیب ذخیره‌شدن در فایل پردازش می‌شوند (آهنگ اول اندیس 0 دارد).

- برای هر آهنگ با اندیس i مراحل زیر انجام می‌شود:
- ابتدا مقدار زیر برای نام آهنگ محاسبه می‌شود:

$$h_{name} = \sum_{j=0}^{|name|-1} (\text{ASCII}(name[j]) \times (j + 1))$$

- سپس مقدار زیر برای نام هنرمند محاسبه می‌شود:

$$h_{artist} = \sum_{j=0}^{|artist|-1} (\text{ASCII}(artist[j]) \times (j + 1))$$

- هاش مربوط به آهنگ برابر است با:

$$songHash = 31 \times h_{name} + 17 \times h_{artist} + 13 \times duration + 7 \times isFavorite$$

- مقدار امضای پلیلیست به صورت زیر به روزرسانی می‌شود:

$$signature \oplus = songHash \ll (i \bmod 16)$$

خروجی

- گر پلیلیست خالی باشد:

```
Playlist signature: 0
```

- در غیر این صورت:

```
Playlist signature: <number>
```

۱۱. عدم قطعیت پلیلیست:

```
playlist_entropy
```

هدف این دستور محاسبه‌ی entropy پلی‌لیست بر اساس توزیع هنرمندان است. entropy معیاری برای اندازه‌گیری میزان تنوع داده‌هاست.

روش محاسبه

فرض کنید:

- پلی‌لیست شامل n آهنگ است
- تعداد هنرمند متفاوت در پلی‌لیست برابر k است
- تعداد آهنگ‌های هنرمند i برابر c_i است احتمال انتخاب تصادفی یک آهنگ از هنرمند i :

$$p_i = \frac{c_i}{n}$$

فرمول entropy:

$$H = - \sum_{i=1}^k p_i \log_2(p_i)$$

خروجی

- اگر پلی‌لیست خالی باشد:

```
Playlist entropy: 0.000
```

- در غیر این صورت:

```
Playlist entropy: <value>
```

عدد با سه رقم اعشار چاپ می‌شود.

۱۲. خروج از برنامه:

```
exit
```

مثال

ورودی نمونه ۱

```
add NovemberRain GunsNRoses 8:57
add Paranoid BlackSabbath 2:48
add Again Archive 16:18
add MyEyesHaveSeenYou TheDoors 2:22
list
star NovemberRain
star paraNOID
unstar Again
star Again
delete BillieJean
delete Paranoid
add HighwayToHell ACDC 3:28
add Nutshell AliceInChains xx:yy
add ThemBones ALiceInChains 2:34
star ThemBones
filter_favorites
sort_duration
list
sort_name
list
sort_artost
sort_artist
playlist_signature
playlist_entropy
exit
```

خروجی نمونه ۱

```
Song added: NovemberRain by GunsNRoses
Song added: Paranoid by BlackSabbath
Song added: Again by Archive
Song added: MyEyesHaveSeenYou by TheDoors
NovemberRain | GunsNRoses | 537 | Normal
Paranoid | BlackSabbath | 168 | Normal
Again | Archive | 978 | Normal
MyEyesHaveSeenYou | TheDoors | 142 | Normal
```

```
Starred: NovemberRain
Unstarred: Again
Starred: Again
Song not found: BillieJean
Deleted: Paranoid
Song added: HighwayToHell by ACDC
Invalid duration format: xx:yy
Song added: ThemBones by ALiceInChains
Starred: ThemBones
NovemberRain | GunsNRoses | 537 | Favorite
Again | Archive | 978 | Favorite
ThemBones | ALiceInChains | 154 | Favorite
Sorted by duration.
MyEyesHaveSeenYou | TheDoors | 142 | Normal
ThemBones | ALiceInChains | 154 | Favorite
HighwayToHell | ACDC | 208 | Normal
NovemberRain | GunsNRoses | 537 | Favorite
Again | Archive | 978 | Favorite
Sorted by name.
Again | Archive | 978 | Favorite
HighwayToHell | ACDC | 208 | Normal
MyEyesHaveSeenYou | TheDoors | 142 | Normal
NovemberRain | GunsNRoses | 537 | Favorite
ThemBones | ALiceInChains | 154 | Favorite
Sorted by artist.
Playlist signature: 10969326
Playlist entropy: 2.322
```

ورودی نمونه ۲

```
add HolyWars Megadeth 06:32
add SymphonyOfDestruction Megadeth 04:02
add TornadoOfSouls Megadeth 05:20
add PeaceSells Megadeth 04:02
add Hangar18 Megadeth 05:11
star HolyWars
star TornadoOfSouls
star Hangar18
filter_favorites
sort_artist
list
unstar TornadoOfSouls
```

```
delete SymphonyOfDestruction
delete NotHere
add PleasePleasePlease SabrinaCarpenter 4:23
add Espresso SabrinaCarpenter 3:21
add LoveBites DefLeppard 5:22
sort_duration
list
playlist_entropy
playlist_signature
exit
```

خروجی نمونه ۲

```
Song added: HolyWars by Megadeth
Song added: SymphonyOfDestruction by Megadeth
Song added: TornadoOfSouls by Megadeth
Song added: PeaceSells by Megadeth
Song added: Hangar18 by Megadeth
Starred: HolyWars
Starred: TornadoOfSouls
Starred: Hangar18
HolyWars | Megadeth | 392 | Favorite
TornadoOfSouls | Megadeth | 320 | Favorite
Hangar18 | Megadeth | 311 | Favorite
Sorted by artist.
Hangar18 | Megadeth | 311 | Favorite
HolyWars | Megadeth | 392 | Favorite
PeaceSells | Megadeth | 242 | Normal
SymphonyOfDestruction | Megadeth | 242 | Normal
TornadoOfSouls | Megadeth | 320 | Favorite
Unstarred: TornadoOfSouls
Deleted: SymphonyOfDestruction
Song not found: NotHere
Song added: PleasePleasePlease by SabrinaCarpenter
Song added: Espresso by SabrinaCarpenter
Song added: LoveBites by DefLeppard
Sorted by duration.
Espresso | SabrinaCarpenter | 201 | Normal
PeaceSells | Megadeth | 242 | Normal
PleasePleasePlease | SabrinaCarpenter | 263 | Normal
Hangar18 | Megadeth | 311 | Favorite
TornadoOfSouls | Megadeth | 320 | Normal
```

LoveBites | DefLeppard | 322 | Normal
HolyWars | Megadeth | 392 | Favorite
Playlist entropy: 1.379
Playlist signature: 10324849

Json لعنتی

- محدودیت زمان: ۱ ثانیه
- محدودیت حافظه: ۲۵۶ مگابایت

معرفی JSON

یک فرمت متنی سبک برای ذخیره‌سازی و انتقال داده‌هاست (JavaScript Object Notation مخفف) JSON بسیار ساده که به‌طور گسترده در برنامه‌نویسی وب و ارتباط بین سرویس‌ها استفاده می‌شود. ساختار و قابل خواندن برای انسان است و از دو نوع داده‌ی اصلی تشکیل شده

- **آرایه‌ها (Arrays):** لیستی از مقادیر که با `[` شروع و با `]` پایان می‌یابد.
- **اشیاء (Objects):** مجموعه‌ای از جفت‌های کلید-مقدار که با `{` شروع و با `}` پایان می‌یابد.

مثال ساده‌ای از یک رشته‌ی JSON:

```
1 {  
2   "name": "Ali",  
3   "age": 25,  
4   "address" : {  
5     "city" : "Tehran",  
6     "province" : "Tehran"  
7   }  
8 }
```

شرح مسئله

در این سوال، هدف شما پیاده‌سازی یک **JSON decoder** ساده است.

در ابتدا، آدرس یک فایل متنی به شما داده می‌شود که محتوای آن یک رشته‌ی JSON معتبر است. شما باید این رشته را **پارس** کرده و اطلاعات آن را در یک ساختار داده‌ای مناسب (مانند `struct`) ذخیره کنید. سپس، تعدادی **پرس‌وجو (query)** به شما داده می‌شود. شما باید مقدار متناظر با آن query را پیدا کرده و چاپ کنید.

انواع Query

پس از بارگذاری و پارس کردن فایل JSON، تعدادی پرسوجو (Query) به شما داده می‌شود. هر پرسوجو یکی از انواع زیر است:

1. depth

در این نوع پرسوجو، شما باید **عمق شیء JSON** را محاسبه و چاپ کنید. عمق به معنای تعداد لایه‌های تو در توی اشیاء (nested objects) است.

مثال:

```
1 | {  
2 |   "user": {  
3 |     "address": {  
4 |       "zipcode": 12345  
5 |     }  
6 |   }  
7 | }  
8 | در این مثال، عمق برابر با 3 است (`user` → `address` → `zipcode`)
```

2. typeof "key"

در این پرسوجو، نوع داده‌ی مربوط به کلید مشخص شده باید چاپ شود.

انواع داده‌های قابل تشخیص:

- INT
- STRING
- OBJECT

فرمت کلید می‌تواند به صورت ساده یا مسیر تو در تو باشد:

- اگر کلید در سطح اول باشد، فقط نام آن نوشته می‌شود:

age

- اگر کلید در لایه‌های داخلی باشد، مسیر آن با `.` جدا می‌شود:

```
address.postAddress.zipcode
```

3. `sizeof "key"`

در این پرس و جو، مقدار کلید مشخص شده باید چاپ شود.

نکات مهم:

- اگر مقدار از نوع `OBJECT` باشد، به جای چاپ مقدار، باید این پیام چاپ شود:

```
I can not print objects!
```

- اگر کلید مورد نظر وجود نداشته یا مسیر نامعتبر بود، باید این پیام چاپ شود:
`No value found for this address`!! فرمت کلید مشابه `typeof` است و می تواند ساده یا مسیر تو در تو باشد.

4. `exit`

هر وقت این دستور وارد شود، برنامه به اتمام میرسد.

```
exit
```

ورودی نمونه

```
user.json
depth
typeof user.name
sizeof user.age
sizeof user.address
sizeof user.location
exit
```

خروجی نمونه

```
3
STRING
```

25

I can not print objects!

No value found for this address!!